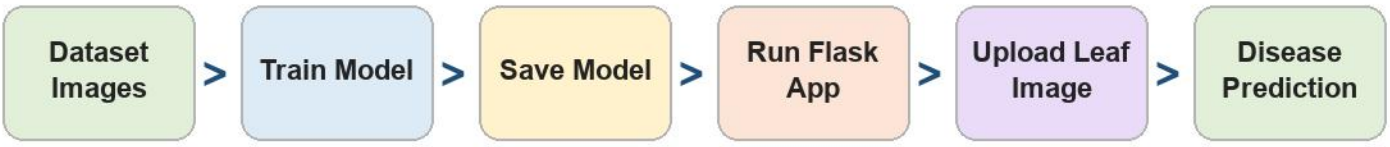
Plant Leaf Disease Detection

Complete Program Execution Flow - beginner friendly ML explanation

This PDF explains how your project runs from dataset/training to the final web app prediction. It uses simple words because you know the basics of machine learning and need the full program flow clearly.

1. Big Picture Flow



Think of the project in two parts:

- ? Training flow: teaches the model using many labelled leaf images.
- ? Execution/prediction flow: uses the trained model inside the web app to classify a new leaf photo.

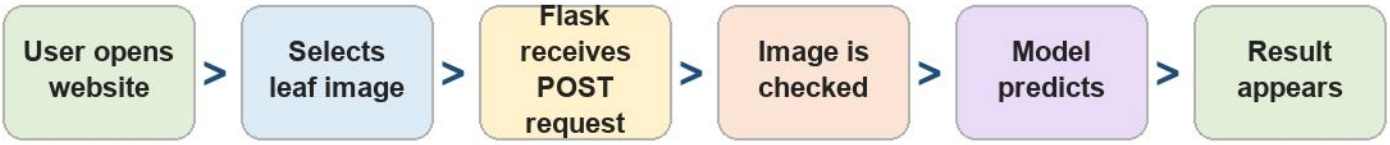
2. Files Used In This Project

File / Folder	Purpose
app.py	Starts the Flask web application and handles image upload requests.
model_utility.py	Loads the saved TensorFlow model, preprocesses the image, predicts disease, and gives treatment tips.
train_mobilenetv2.py	Optional training script. It trains/fine-tunes MobileNetV2 and saves a new model.
Train.ipynb	Notebook version used for training/experiments and model analysis.
templates/index.html	Main web page where the user uploads a leaf image and views results.
templates/manual.html	Shows supported crops and disease classes.
models/plant_leaf_disease_detector	Saved TensorFlow model used by the app for prediction.
plots/	Images showing model structure, accuracy/loss, and confusion matrix.
requirements.txt	Python libraries needed to run the project.

3. How To Execute The Web App

1. Open PowerShell or terminal in the project folder: E:\project\PlantLeafDisease
2. Install dependencies once using: pip install -r requirements.txt
3. Make sure the saved model exists at: models/plant_leaf_disease_detector
4. Run the app using: python app.py
5. Open browser and go to: http://127.0.0.1:5000
6. Upload a clear JPG/PNG/WEBP leaf image and click Analyze Leaf.

4. Web App Prediction Flow



Step	What Happens	Main Code
1	Browser opens the upload form.	app.py route / and templates/index.html
2	User uploads a leaf image.	HTML form sends file named leaf_image
3	App checks file type and size.	allowed_file(), MAX_CONTENT_LENGTH
4	Image is opened and converted to RGB.	PIL Image.open(...).convert("RGB")
5	Image is resized to 256x256 and converted to numbers.	preprocess_image() in model_utility.py
6	Saved TensorFlow model predicts probabilities for 38 classes.	predict_leaf_disease()
7	Highest probability class becomes final answer.	np.argmax(probabilities)
8	Crop, disease, confidence, alternatives, and recommendations are displayed.	index.html result section

5. What Happens Inside The ML Model

A computer cannot directly understand a leaf photo like a human. The program converts the image into numbers, then the neural network compares patterns such as color, spots, edges, and leaf texture.

? Input image size: 256 x 256 pixels with 3 color channels: RGB.

? Output: probability scores for all supported crop-disease classes.

? The class with the highest score is shown as the prediction.

? Confidence is useful, but the result should still be checked with a clear image and supported crop.

6. Optional Training Flow

Training Stage	Simple Meaning
Load dataset	Reads images from train/ and valid/ folders. Folder names become class labels.
Preprocess images	Resizes images and converts pixel values into the format expected by MobileNetV2.
Use MobileNetV2	Starts from a model already trained on ImageNet, so it understands basic visual features.
Train classifier head	Learns which visual patterns match each plant disease class.
Fine-tune model	Unfreezes later layers so the model adapts better to leaf disease images.
Save model	Exports the trained model into models/plant_leaf_disease_detector for the Flask app.

7. Important ML Words In Simple Language

Term	Meaning
Class	One output category, such as Potato Early blight or Apple healthy.
Training data	Images used to teach the model.
Validation data	Images used to check whether the model is learning correctly.
Preprocessing	Preparing the image before prediction: RGB conversion, resizing, and scaling pixel values.
Softmax	Turns model output numbers into probabilities that add up to 1.
Confidence	The probability score of the selected prediction. Higher is usually better, but not perfect.
Overfitting	When a model memorizes training images instead of learning general patterns.

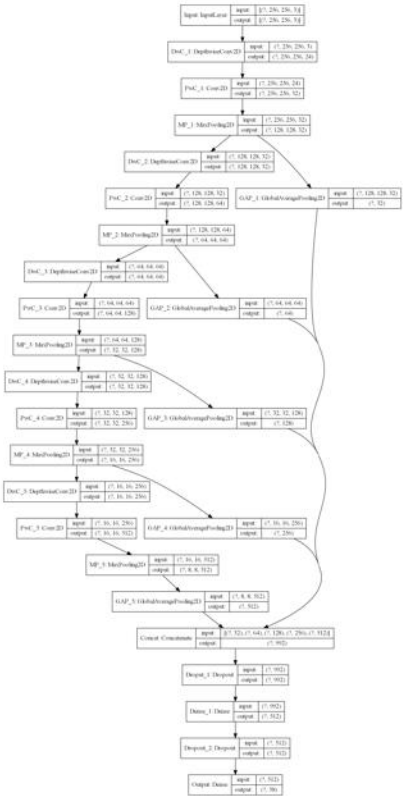
8. Supported Output Information

? Predicted crop name, for example Tomato, Potato, Apple, Grape, Corn, etc.

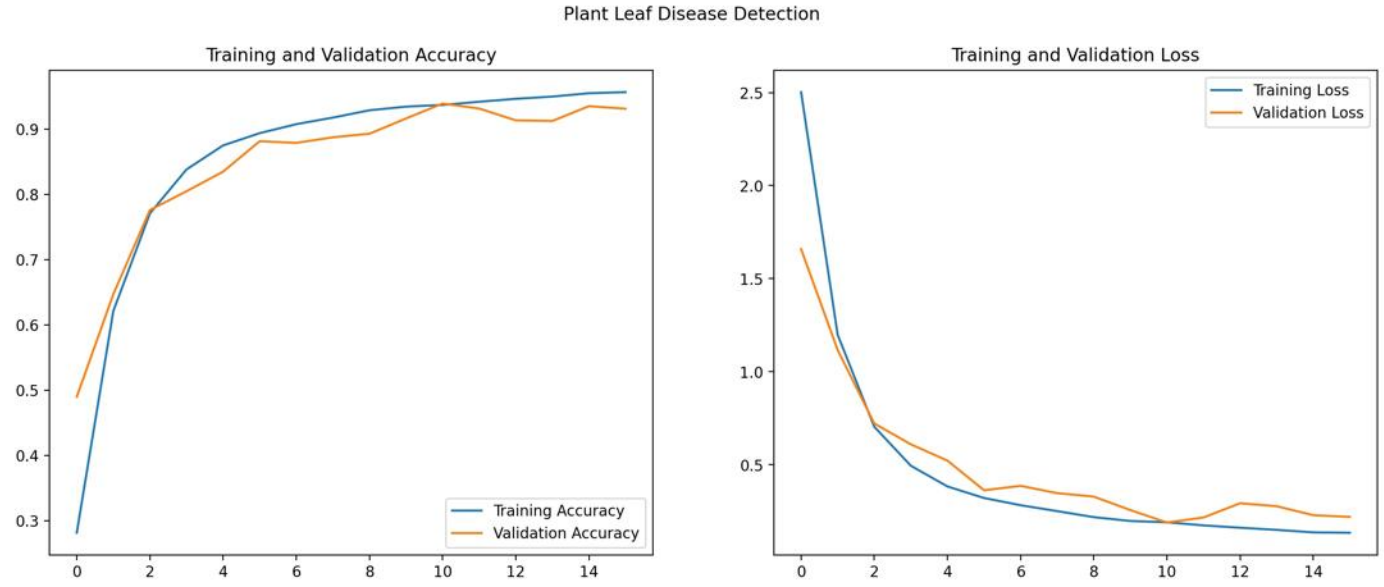
- ? Predicted condition/disease, for example healthy, Early blight, Late blight, Powdery mildew, etc.
- ? Confidence percentage.
- ? Other possible predictions, useful when confidence is not very high.
- ? Treatment recommendations from model_utility.py.

9. Diagrams From The Project

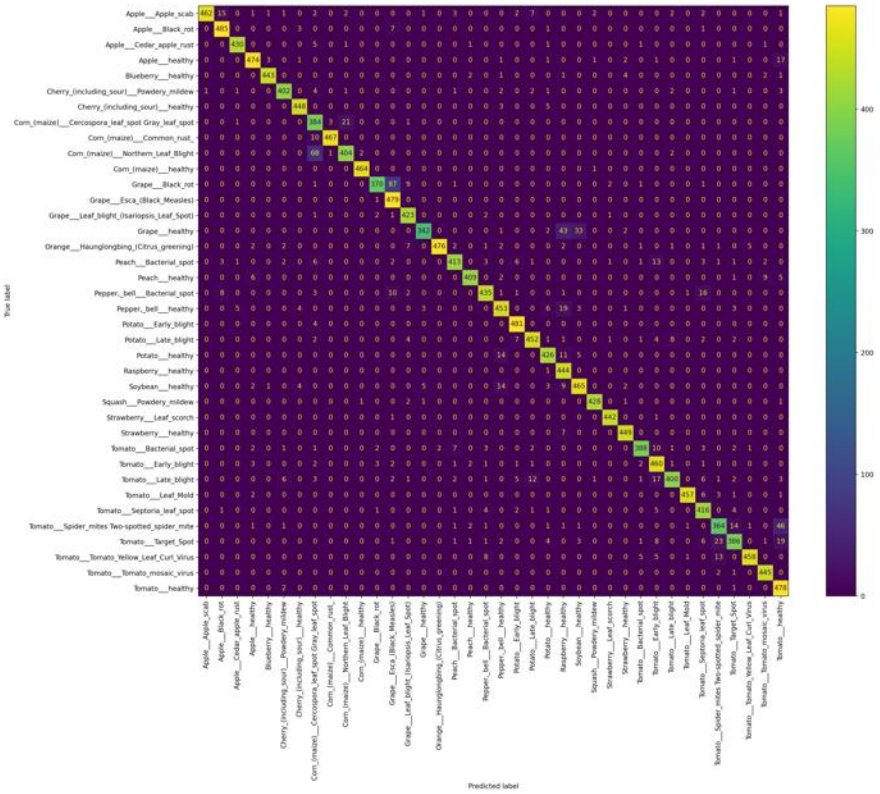
Model Architecture Diagram



Accuracy And Loss During Training



Validation Confusion Matrix



10. Quick Troubleshooting

Problem	Reason / Fix
TensorFlow error	Install requirements again or check Python/TensorFlow compatibility.
Model not found	Check that models/plant_leaf_disease_detector contains saved_model.pb and variables folder.
Upload rejected	Use JPG, JPEG, PNG, or WEBP and keep file below 8 MB.
Wrong prediction	Use a clear single-leaf image with good lighting; the crop must be supported.
Training is slow	Deep learning training needs high RAM/GPU. Use the saved model for normal demo execution.

Final Summary

For demonstration, you normally do not need to train again. Run app.py, upload a leaf image, and the already saved TensorFlow model will predict the crop disease. Training is only needed if you want to build a new model or improve the existing one with more data.